



Trailhead



[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)



[Get Started with Apex](#)

Get Started with Apex

Learning Objectives

After completing this unit, you'll be able to:

- Describe the key features of the Apex programming language.
- Save an Apex class and call methods with Anonymous.Apex.
- Use the Developer Console to inspect debug logs.

Before You Begin

We're excited to accompany you on your Apex adventure. While this module covers basic information about the programming language, it also goes deep, quickly. If this module is your first exposure to Apex, we highly recommend that you first go through the [Quick Start: Apex](#)

(<https://trailhead.salesforce.com/content/learn/projects/quickstart-apex>) project. Then come back. We'll be waiting for you!

What is Apex?

Apex is a programming language that uses Java-like syntax and acts like database stored procedures. Apex enables developers to add business logic to system events, such as button clicks, updates of related records, and Visualforce pages.

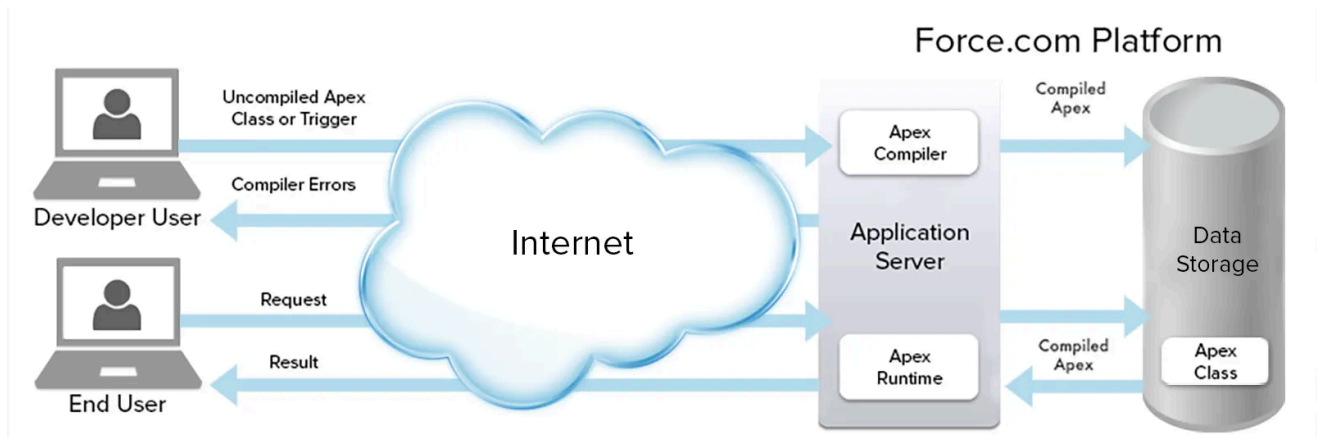
As a language, Apex is:

- **Hosted**—Apex is saved, compiled, and executed on the server—the Salesforce Platform.
- **Object-oriented**—Apex supports classes, interfaces, and inheritance.
- **Strongly-typed**—Apex validates references to objects at compile time.
- **Multitenant-aware**—Because Apex runs on a multitenant platform, it guards closely against runaway code by enforcing limits, which prevent code from monopolizing shared resources.





- **Easy to use**—Apex is based on familiar Java idioms.
 - **Easy to test**—Apex provides built-in support for unit test creation, execution, and code coverage. Salesforce ensures that all custom Apex code works as expected by executing all unit tests prior to any platform upgrades.
 - **Versioned**—Custom Apex code can be saved against different versions of the API.
- Get Started with Apex



Apex Language Highlights

Like other object-oriented programming languages, these are some of the language constructs that Apex supports:

- Classes, interfaces, properties, and collections (lists, maps, and sets).
- Object and array notation.
- Expressions, variables, and constants.
- Conditional statements (if-then-else) and control flow statements (for loops and while loops).

Unlike other object-oriented programming languages, Apex supports:

- Cloud development as Apex is stored, compiled, and executed in the cloud.
- Triggers, which are similar to triggers in database systems.
- Database statements that allow you to make direct database calls and query languages to query and search data.
- Transactions and rollbacks.
- The `global` access modifier, which is more permissive than the `public` modifier and allows access across namespaces and applications.
- Versioning of custom code.

In addition, Apex is a case-insensitive language.



You can also write Apex and access debugging information directly in the browser by using the Salesforce user interface. Open the Developer Console under Your Name or Apex Basics & Database > /content/learn/modules/apex_database_the_quick_access_menu/8trailmix_slug=salesforce-platform-developer).

Get Started with Apex

Data Types Overview

Apex supports various data types, including a data type specific to Salesforce—the sObject data type.

Apex supports the following data types.

- A primitive, such as an Integer, Double, Long, Date, Datetime, String, ID, Boolean, among others.
- An sObject, either as a generic sObject or as a specific sObject, such as an Account, Contact, or MyCustomObject__c (you'll learn more about sObjects in a later unit.)
- A collection, including:
 - A set of primitives, sObjects, user defined objects, objects created from Apex classes, or collections
 - A map from a primitive to a primitive, sObject, or collection
- A typed list of values, also known as an **enum**
- User-defined Apex classes
- System-supplied Apex classes

Apex Collections: List

Lists hold an ordered collection of data of the same type.

The following two declarations are equivalent. The `colors` variable is declared using the List syntax.

```
1 List<String> colors = new List<String>();
```



Alternatively, the `colors` variable can be declared using array notation.

```
1 String[] colors = new List<String>();
```





This example shows how to add elements to a list when you create it, and then use the `add()`  method to add more elements.

[Apex Basics & Database \(/content/learn/modules/apex_database?](#)

[trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Get Started with Apex

```
1 // Create a list and add elements to it in one step
2 List<String> colors = new List<String> { 'red', 'green', 'blue'
3 };
4 // Add elements to a list after it has been created
5 List<String> moreColors = new List<String>();
6 moreColors.add('orange');
  moreColors.add('purple');
```



List elements can be read by specifying an index between square brackets, known as array notation. Also, you can use the `get()`  method to read a list element. This example is based on the lists created in the previous example and shows how to read list elements using either method. The example also shows how to iterate over list elements.

```
1 // Get elements from a list
2 String color1 = moreColors.get(0);
3 String color2 = moreColors[0];
4 System.assertEquals(color1, color2);
5 // Iterate over a list to read elements
6 System.debug('Print out the colors in moreColors:');
7 for(String color : moreColors) {
8     // Write value to the debug log
9     System.debug(color);
10 }
```



Beyond the Basics

Apex supports two other collection types: Set and Map. You can learn more about these in the [Collections \(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/langCon_apex_collections.htm\)](https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/langCon_apex_collections.htm) section of the Apex Developer Guide.



Save an Apex Class

Apex Basics & Database (/content/learn/modules/apex_database?

Save the EmailManager copy class in your organization.
trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer).

Get Started with Apex

1. Open the Developer Console under Your Name or the quick access menu (⚙️).
2. In the Developer Console, click **File | New | Apex Class**, and enter EmailManager for the class name, and then click **OK**.
3. Replace the default class body with the EmailManager copy class example.

The EmailManager copy class has a public method (sendMail() copy) that sends email and uses built-in Messaging methods of the Apex class library. Also, this class has a private helper method (inspectResults() copy), which can't be called externally because it is private but is used only within the class. This helper method inspects the results of the email send call and is called by sendMail() copy.

```
1 public class EmailManager {
2     // Public method
3     public void sendMail(String address, String subject,
4 String body) {
5         // Create an email message object
6         Messaging.SingleEmailMessage mail = new
7 Messaging.SingleEmailMessage();
8         String[] toAddresses = new String[] {address};
9         mail.setToAddresses(toAddresses);
10        mail.setSubject(subject);
11        mail.setPlainTextBody(body);
12        // Pass this email message to the built-in sendEmail
13 method
14        // of the Messaging class
15        Messaging.SendEmailResult[] results =
16 Messaging.sendEmail(
17             new
18 Messaging.SingleEmailMessage[] { mail });
19        // Call a helper method to inspect the returned
20 results
```



Show More



Beyond the Basics

Apex Basics & Database (/content/learn/modules/apex_database?

trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

The class you just saved makes use of object-oriented programming (OOP). The class encapsulates the methods that are related to managing email. To be a perfect example Get Started with Apex would also contain member variables (attributes) and accessor methods to access those attributes, but for simplicity our class doesn't have these.

Call a Method to Send an Email

Let's invoke the public method. We'll use anonymous Apex execution to do so. Anonymous Apex allows you to run lines of code on the fly and is a handy way to invoke Apex, especially to test out functionality. Debug log results are generated, as with any other Apex execution.



There are other ways to invoke Apex, for example, through triggers. You'll learn more about triggers in another module.

1. In the Developer Console, click **Debug | Open Execute Anonymous Window**.
2. In the window that opens, enter the following. Replace 'Your email address' with your email address.

```
1 EmailManager em = new EmailManager();  
2 em.sendMail('Your email address', 'Trailhead Tutorial', '123  
body');
```



3. Click **Execute**.

You should now receive an email in your inbox. Check your email!

Inspect Debug Logs

Debug logs are useful for debugging your code. When Apex methods execute, the calls are logged in the debug log. Also, you can write your own debug messages to the log, which helps in debugging your code in case there are errors. The `inspectResults()`  helper method, which is called by `sendMail()` , writes messages to the log by using the `System.debug()`  method to indicate whether the email send operation was successful or had errors. You can look for these messages in the debug log that was generated when you executed the method.



Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Get Started with Apex

Timestamp	Event	Details
17:11:54:463	USER_DEBUG	[30][DEBUG]Email sent successfully

Logs	Tests	Checkpoints	Query Editor	View State	Progress	Problems
Operation	Time	Status	Read	Size		
Test User 0000000000001007 /services/data/v33.0/tooling/executeAnonym...	11/10/2014, 5...	Success		8.13 KB		

You'll see the following message in the filtered log view, assuming the email was sent without errors.

1 | `DEBUG|Email sent successfully`



Also, you can filter the debug log by searching for any keyword in the **Filter** field, or by selecting any of the other options. For more

information, see [Log Inspector](#)

([https://help.salesforce.com/s/articleView?](https://help.salesforce.com/s/articleView?id=platform.code_dev_console_view_system_log.htm&language=en_US)

[id=platform.code_dev_console_view_system_log.htm&language=en_US](https://help.salesforce.com/s/articleView?id=platform.code_dev_console_view_system_log.htm&language=en_US)).

Call a Static Method

Because the `sendMail()` method in our class doesn't access class member variables, it doesn't need to be an instance method. Let's change it to a static method by adding the `static` keyword to its declaration. Static methods are easier to call than instance methods because they don't need to be called on an instance of the class but are called directly on the class name.

1. In the Developer Console, find the open tab for the `EmailManager` class and modify the first line of the `sendMail()` method definition to the following (the only change is the added `static` keyword.)

```
1 public static void sendMail(String address, String subject,
  String body) {
```



2. Click **File | Save** to save your class.
3. Modify the statements in your Execute Anonymous window to call the static method on the class name.



and optionally, the debug log as in the previous steps.

[Apex Basics & Database \(/content/learn/modules/apex_database?](#)

[trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)

Resources

[Get Started with Apex](#) [Apex Developer Guide: Introducing Apex](#)

(https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/apex_intro.htm)

- [Apex Developer Guide: Loops](#) (https://developer.salesforce.com/docs/atlas.en-us.apexcode.meta/apexcode/langCon_apex_loops.htm)
- [Trailhead: Developer Console Basics](#) (https://trailhead.salesforce.com/content/learn/modules/developer_console)

GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

YOUR CHALLENGE

Create an Apex class with a method that returns a list of strings

Create an Apex class with a method that returns a list of formatted strings. The length of the list is determined by an integer parameter.

- The Apex class must be called `StringListTest` [copy](#) and be in the public scope
- The Apex class must have a public static method called `generateStringList` [copy](#)
- The **generateStringList** method must return a list of strings
 - The method must accept an incoming `Integer` [copy](#) as a parameter, which will be used to determine the number of returned strings
 - The method must return a list of strings. Each element in the list must have the format **Test n**, where `n` [copy](#) is the index of the current string in the list. For example, if the input is **3**, then the output should be **['Test 0', 'Test 1', 'Test 2']**. Remember that in Apex, the index position of the first element in a list is always `0` [copy](#) .



[Apex Basics & Database \(/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer\)](#)
Check Challenge to Earn 500 Points

Get Started with Apex

Learn

Badges
Trails
Trailmixes
Superbadges
Trailhead Academy
Career Paths

Certifications

Certifications
Maintain Certifications
Verify Certifications

Community

Trailblazer Community
Events
Agentblazers
Salesforce Developers
Salesforce Admins
Code of Conduct
Report Illegal Content (EU)

Extras

Trailhead Login
Sales Enablement
Trailblazer Store
Salesforce Help

Download Trailhead GO



© 2026 Salesforce, Inc. All rights reserved.

[Privacy Statement](#) [Terms of Use](#) [Use of Cookies](#) [Trust](#) [Accessibility](#) [Cookie Preferences](#)

Your Privacy Choices

Engli